Real-Time Face and Object Recognition in Python with Lightweight Neural Networks and Three-Sigma Open-Set Thresholds

Margaret Gonzalez
Computer Science Graduate Program
Full Sail University
Winter Park, Florida, United States
mdgonzalez1@student.fullsail.edu

***Abstract*: This paper presents a Python application that recognizes everyday objects, including school buses and cell phones, detects faces, and names enrolled people in real time on an ordinary processor. Objects are detected by a YOLOv10-N network covering 80 classes, a MobileNetV3 classifier separates school buses from other buses and trucks, YuNet locates faces and five landmarks, and SFace converts each aligned face into a 128-value embedding that is compared with an enrolled gallery by cosine similarity. The acceptance threshold is set with three-sigma control limits, adapted from earlier neural network work on age and gender prediction, computed on the best-match scores of 500 background faces. On the 5,000 COCO validation images, the detector reached 38.0% mean average precision at a 640-pixel input and 28.5% at 320 pixels while running 3.7 times faster. Face verification reached 99.25% accuracy on the 6,000 Labeled Faces in the Wild pairs, YuNet found 99% of masked faces against 43% for a Haar cascade, and the calibrated threshold named all 100 probes of 20 enrolled people while rejecting all 200 strangers, whereas the default threshold accepted 7% of them.**

Keywords: object detection, face detection, face recognition, YOLO, open-set identification, three-sigma control limits, real-time computer vision

I. Introduction

A camera application that can tell a school bus from a city bus, notice a cell phone in a hand, and greet a known person by name combines two related problems. Object recognition must find many kinds of objects of very different sizes in one pass, and face recognition must decide whether a face belongs to someone who has been enrolled or to a stranger. Both problems are now solved mainly with convolutional neural networks, but most published systems assume a graphics processing unit, while a student project or a small business usually runs on a laptop processor.

The goal of this project was therefore an application that runs on a central processing unit (CPU) with only three Python packages, recognizes the objects named in the assignment, detects and recognizes faces, and comes with measured evidence of its accuracy and speed. The contributions are a two-branch pipeline that adds a fine-grained school bus check to a general detector, an open-set face threshold based on three-sigma control limits, and an evaluation on public benchmarks that covers detection accuracy, robustness, open-set identification, and the effect of each optimization.

II. Background and Related Work

The You Only Look Once (YOLO) family predicts boxes and classes for a whole image in a single network pass. Terven et al. trace its development through many versions and describe the common evaluation metrics and the non-maximum suppression (NMS) step that removes duplicate boxes [1]. YOLOv10 introduced consistent dual assignments so that the network can be trained without NMS, and its small model is 1.8 times faster than the RT-DETR-R18 detector at similar accuracy [2]. Its nano version, YOLOv10-N, has 2.3 million parameters and reported 38.5% average precision on COCO [2]. Detectors of this kind are limited to the classes they were trained on; open-vocabulary detectors such as YOLO-World accept text prompts instead and reached 35.4 average precision on the LVIS benchmark at 52.0 frames per second on a V100 graphics card [3].

For faces, Feng et al. survey the move from the Viola-Jones cascade to efficient convolutional detectors and compare them by accuracy, operations, and latency [4]. YuNet is one of the smallest of these detectors, with 75,856 parameters, 81.1% average precision on the hard subset of WIDER FACE, and a latency of about 1.6 milliseconds [5]. Recognition then maps an aligned face to an embedding. SFace trains such embeddings with a sigmoid-constrained hypersphere loss that limits how strongly noisy training images are pulled together or pushed apart [6]. Gururaj et al. review face recognition techniques and identify illumination, pose, expression, occlusion, and aging as the main remaining challenges [7].

Deciding when a score is high enough is a separate problem. Dileep and Danti predicted age group and gender with a feed-forward neural network and then used three-sigma control limits, the mean plus or minus three standard deviations of the network output, to decide whether a result was reliable or a misdetection; they reported a 95% success rate on 300 test images [8]. This project applies the same statistical idea to similarity scores. Finally, face recognition carries privacy and security risks. Laishram et al. identify real-time identification without consent as the most severe privacy concern and survey privacy-preserving methods [9], and Leyva et al. categorize the attack vectors that threaten face recognition systems [10].

III. System Design

The application is written in Python and uses OpenCV for image input, YuNet, SFace, and drawing; ONNX Runtime for the YOLOv10-N and MobileNetV3 networks; and NumPy for the mathematics. All models are stored as files in the project, so no network connection or deep learning framework is required at run time. Fig. 1 shows the two branches that process each camera frame, and Table I lists the models.

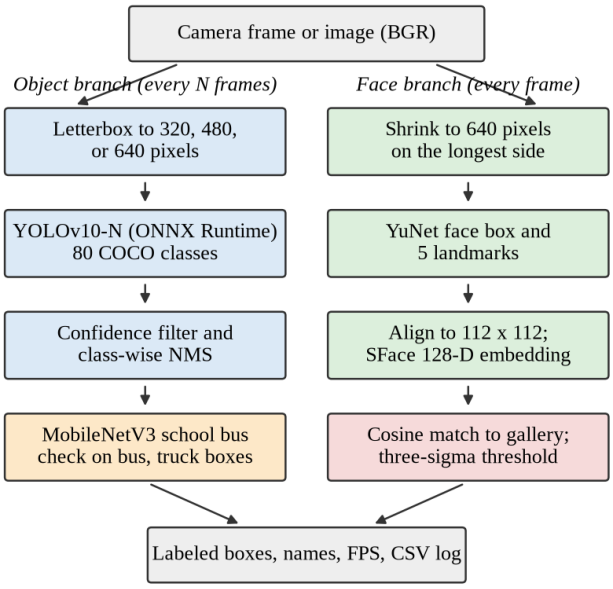


Fig. 1. Processing pipeline. The object branch can run on every Nth frame; the face branch runs on every frame.

TABLE I. MODELS USED BY THE APPLICATION

Task	Model	Input	Size	Data
Objects, 80 classes	YOLOv10-N [2]	320-640 px	9.7 MB	COCO
School bus check	MobileNetV3-Large [11]	224 px	21.9 MB	ImageNet
Face detection	YuNet [5]	frame	0.2 MB	WIDER FACE
Face embedding	SFace [6]	112 px	38.7 MB	Faces [6]
Baseline detector	Haar cascade [4]	frame	0.9 MB	classical

A. Object Detection

Each frame is letterboxed: it is resized without changing its aspect ratio and padded with gray to a square of side S, which is 320, 480, or 640 pixels. For a frame of height h and width w, the scale and padding are

$$s = \min(S/h, S/w),\ \ p_x = (S - sw)/2,\ \ p_y = (S - sh)/2. \quad (1)$$

The network returns 8,400 candidate boxes at 640 pixels, each with a center, width, height, and 80 class scores. The exported graph keeps the dense one-to-many output, so the application keeps candidates whose best class score is above the confidence threshold (0.35 on screen) and maps each box back to the frame with

$$x_1 = (c_x - b_w/2 - p_x)/s,\ \ y_1 = (c_y - b_h/2 - p_y)/s. \quad (2)$$

Duplicate boxes are then removed separately for each class by NMS, which discards a box when its intersection over union (IoU) with a higher-scoring box of the same class exceeds 0.5:

$$\mathrm{IoU}(A, B) = |A \cap B| / |A \cup B|. \quad (3)$$

B. School Bus Refinement

COCO, the data set behind the detector, has a single bus class, so a detector alone cannot name a school bus. ImageNet, however, has a separate school bus class (index 779). The refiner crops every box labeled bus or truck with a 5% margin, resizes it to 224 pixels, and runs MobileNetV3-Large [11], whose softmax output gives the probability of each class k from the logits z:

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j). \quad (4)$$

When the school bus probability is at least 0.30, the box is relabeled school bus. Trucks are included because the detector sometimes boxes a school bus as a truck, as Section V shows.

C. Face Detection, Alignment, and Embedding

YuNet returns each face box with five landmarks: the eye centers, the nose tip, and the mouth corners [5]. SFace uses the landmarks to warp the face to a standard 112 by 112 pixel crop and produces a 128-value embedding, which the application scales to unit length [6]. Two faces are compared with the cosine similarity

$$\cos(a, b) = a \cdot b / (\|a\| \|b\|), \quad (5)$$

which ranges from −1 to 1 and equals the dot product for unit vectors. During enrollment, 15 samples are captured automatically from the webcam while the person turns slightly. A face in a later frame receives the best-match score over all stored samples g_{ij} of every enrolled person i, and the name of that person is shown only when the score reaches the threshold T:

$$m(q) = \max_{i,j} \cos(q, g_{ij}),\ \ \text{accept if } m(q) \geq T. \quad (6)$$

D. Three-Sigma Threshold

Following Dileep and Danti [8], the threshold is a control limit. For genuine scores (two samples of the same person) with mean μ_G and standard deviation σ_G, and impostor scores (different people) with mean μ_I and standard deviation σ_I, the limits are

$$\mathrm{UCL}_I = \mu_I + 3\sigma_I,\ \ \mathrm{LCL}_G = \mu_G - 3\sigma_G,\ \ T = \mathrm{UCL}_I. \quad (7)$$

If impostor scores were normally distributed, only about 0.13% would exceed the upper limit. The important design choice is which impostor scores to use. The first version used every pairwise comparison, as in a one-to-one verification test. The final version scores each of 500 background faces from Labeled Faces in the Wild (LFW) exactly as (6) would score a stranger, by its best match over the whole gallery, so the limit describes the scores that a stranger actually receives. The threshold is recalculated whenever someone is enrolled, and only embeddings, not photographs, are saved.

E. Application Interface

The application accepts a webcam, an image, a folder, or a video. It draws colored object boxes with confidences, green face boxes with the name and similarity of enrolled people, red boxes for unknown faces, and a frames per second (FPS) counter. Keys save screenshots, enroll the largest visible face, toggle the Haar cascade for a side-by-side comparison, and adjust the confidence threshold. Every session is logged to a comma-separated values file with the detections, names, and the time taken by each stage. Twenty-one unit and integration tests check the letterbox and box mathematics, NMS, the control limits, gallery storage, and the complete pipeline on sample photographs.

IV. Evaluation Method

Object detection was measured on all 5,000 images of the COCO 2017 validation set with the official COCO evaluation code at a confidence threshold of 0.001. The main metric is mean average precision (mAP), the precision averaged over 101 recall levels r, 80 classes C, and ten IoU thresholds from 0.50 to 0.95 [1]:

$$\mathrm{mAP} = (1/10|C|)\,\Sigma_t\,\Sigma_c\,(1/101)\,\Sigma_r\,p_{c,t}^{\mathrm{interp}}(r). \tag{8}$$

School bus recognition was measured on ImageNetV2, a test set collected after ImageNet to check how classifiers generalize to new images [12]. Its ten school bus photographs were the positives, and 100 photographs of ten similar vehicles, such as trolleybuses, minibuses, and streetcars, were the negatives. Its ten cellular telephone photographs tested cell phone detection.

Face verification used the 6,000 LFW pairs in the standard ten folds, with thresholds chosen on nine folds and tested on the tenth. Detector robustness was measured on 300 LFW photographs, one per person, under 12 controlled changes, and false alarms were counted on 300 COCO images that contain no people. The open-set test enrolled 20 LFW people with five photographs each, probed them with 100 different photographs, and presented 200 strangers; the 500 background faces used for calibration appeared in no other group. All timings were measured on a two-core virtual machine without a graphics card, so they represent a slow laptop.

V. Results

A. Object Detection

Table II and Fig. 2 show the accuracy and speed trade-off. YOLOv10-N at 640 pixels reached 38.0% mAP, close to the 38.5% reported by its authors [2], in 47 milliseconds per image. Reducing the input to 480 pixels cost 3.0 points, and 320 pixels cost 9.5 points but ran in 13 milliseconds. The larger YOLOv10-S reached 44.6% but needed 105 milliseconds. Fig. 3 shows that buses, which are usually large, kept most of their precision at 320 pixels (AP 51.2 against 60.7), while small objects suffered most: cell phone AP fell from 26.8 to 20.1.

TABLE II. Object Detection on COCO val2017 (5,000 Images)

Model and input	mAP (%)	mAP50 (%)	Bus AP	Phone AP	ms/ img
YOLOV10-N, 320	28.5	42.3	51.2	20.1	13
YOLOV10-N, 480	35.0	51.2	58.8	24.0	28
YOLOV10-N, 640	38.0	54.8	60.7	26.8	47
YOLOV10-S, 640	44.6	62.8	68.6	35.6	105

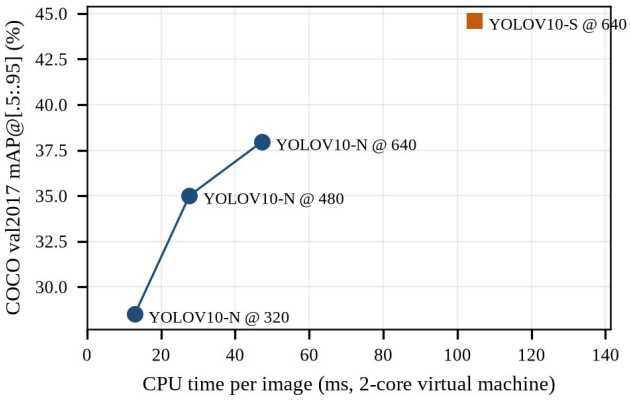


Fig. 2. Accuracy against CPU time per image for each detector configuration.

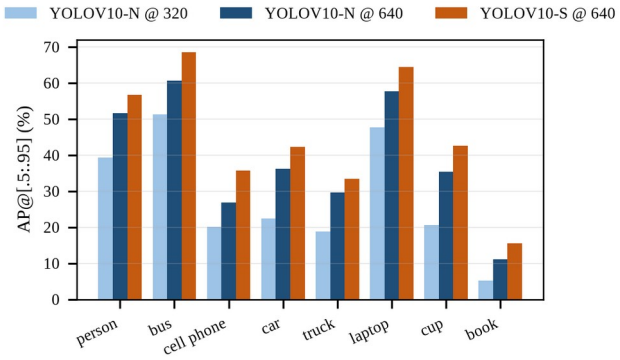


Fig. 3. Average precision for the assignment classes and other common objects.

B. School Bus and Cell Phone Recognition

Calling every detected bus a school bus gave 22.5% precision, because trolleybuses, minibuses, and streetcars are also buses to COCO. Adding the refiner on bus boxes raised precision to 90% with 90% recall. The one missed school bus had been boxed as a truck, so the refiner was extended to truck boxes, which raised recall to 100% at 90.9% precision; the only false positive was a streetcar. Fig. 4 shows that thresholds from 0.30 to 0.45 gave the best F1 score, and 0.30 was chosen to favor recall. The detector found a cell phone in 70% of the ten telephone photographs, which is consistent with the lower average precision of small objects in Fig. 3.

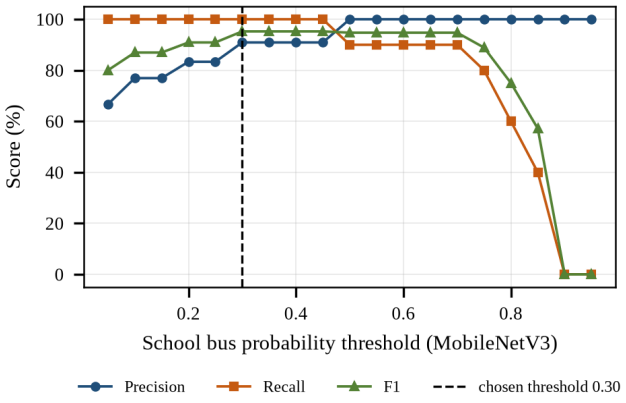


Fig. 4. School bus precision, recall, and F1 score against the MobileNetV3 probability threshold (bus and truck boxes).



Fig. 5. Output of the application on sample photographs: a school bus, a city bus, and cell phones with detected faces.

C. Face Detection Robustness

Fig. 6 compares the two face detectors. Both found nearly every unchanged face, but the Haar cascade dropped to 10% at a 30 degree rotation, 62% with the eyes covered, and 43% with a mask, while YuNet kept 99%, 99%, and 99%. YuNet failed mainly at 45 degrees (79%). On the 300 images without people, the cascade reported 58 false faces and YuNet 3, and YuNet took 12 milliseconds per 640 by 480 image against 112 for the cascade.

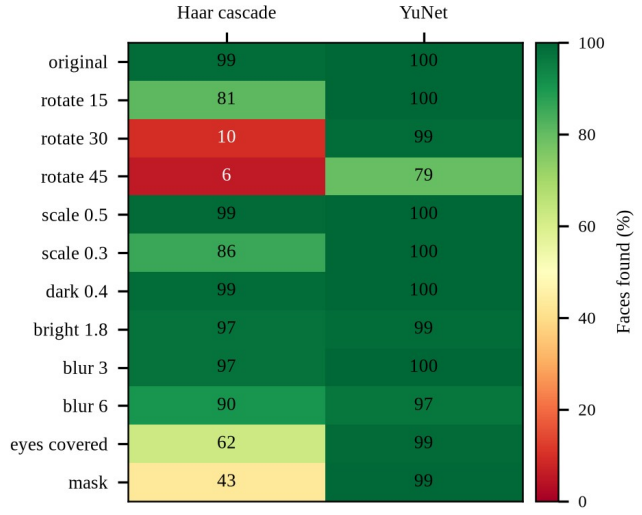


Fig. 6. Percentage of LFW faces found by each detector under rotation, downscaling, brightness, blur, and occlusion.

D. Face Verification and Thresholds

SFace separated the LFW pairs well, with an area under the receiver operating characteristic curve of 0.9988 and a true accept rate of 98.7% at a false accept rate of 0.1%. Fig. 7 shows the score distributions: impostor scores had a mean of 0.084 and an upper control limit of 0.353, and genuine scores had a mean of 0.659 and a lower limit of 0.317. Because the two limits overlap slightly, a few pairs fall in a zone where no threshold is perfect. Table III shows that the three-sigma limit, computed only from impostor scores, reached 99.25% accuracy, within 0.12 points of the threshold tuned for accuracy on labeled pairs and with a lower false accept rate.

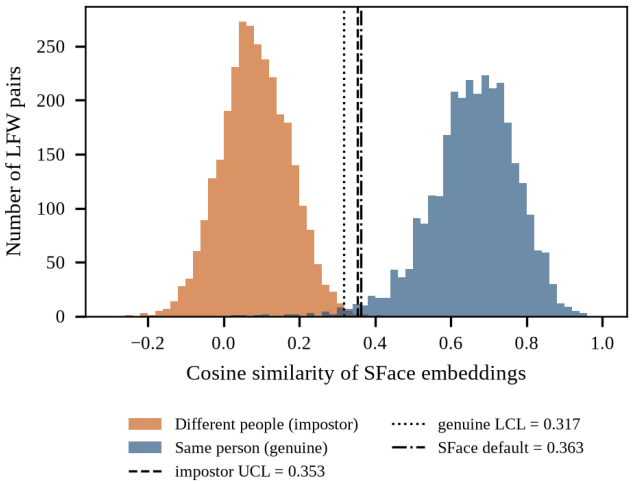


Fig. 7. Genuine and impostor cosine similarity on 6,000 LFW pairs with the three-sigma control limits.

E. Open-Set Identification

Verification compares one pair at a time, but the application compares each face with every stored sample. Fig. 8 shows the consequence. With the default SFace threshold of 0.363, all enrolled people were named correctly, but 7% of strangers were given someone's name, and the pairwise three-sigma limit of 0.350 was worse at 10%. The best-match limit of 0.425 (impostor mean 0.288, standard deviation 0.045) rejected every stranger while still naming all 100 probes correctly, as the confusion matrix in Fig. 9 shows.

TABLE III. THRESHOLD COMPARISON

Threshold	Value	LFW acc. (%)	Stranger FAR (%)	ID rate (%)
SFace default	0.363	99.22	7	100
Best on LFW folds	0.329	99.37	n/a	n/a
Pairwise three-sigma	0.350	99.25	10	100
Best-match three-sigma	0.425	n/a	0	100

[a] LFW accuracy uses ten-fold cross-validation; the pairwise limit on LFW is recomputed on each nine-fold training split. FAR is the false accept rate.

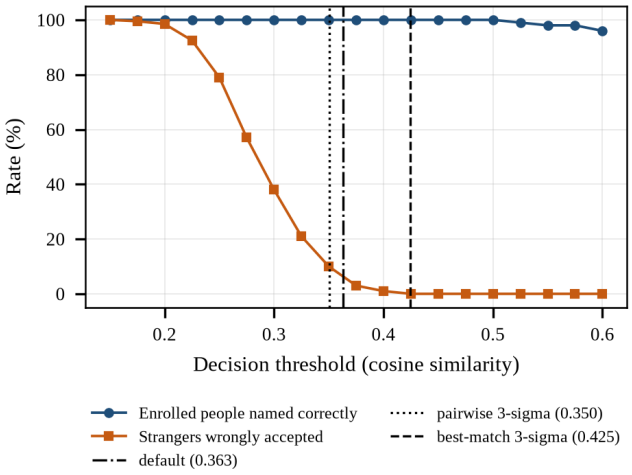


Fig. 8. Open-set identification rate and stranger false accept rate against the decision threshold.

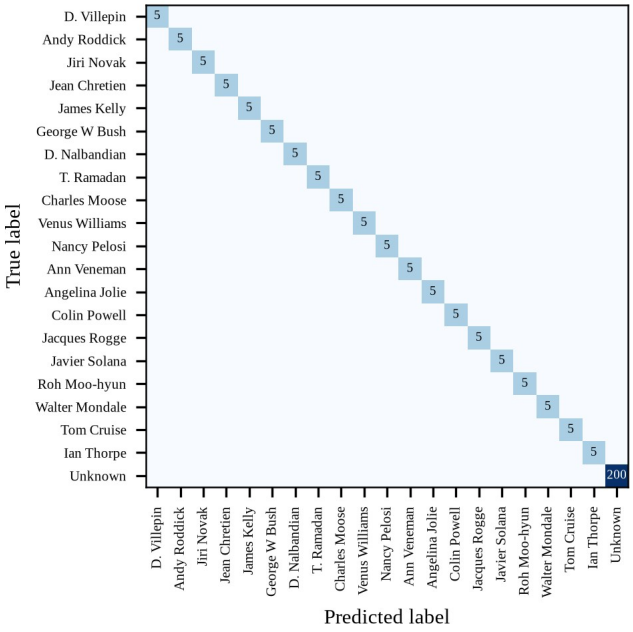


Fig. 9. Confusion matrix for 20 enrolled people and 200 strangers with the best-match three-sigma threshold (counts, logarithmic color scale).

VI. Optimization Approach

Speed was improved one stage at a time while each change was checked against the accuracy results above. Table IV lists the measured time of each stage for a 1280 by 720 webcam frame.

TABLE IV. Stage Times on a Two-Core CPU (1280 by 720 Frame)

Stage	Mean (ms)	Max FPS
YOLOv10-N objects, input 320	13.4	75
YOLOv10-N objects, input 480	27.9	36
YOLOv10-N objects, input 640	46.0	22
MobileNetV3 school bus check (per bus box)	5.6	179
YuNet faces, full 1280 x 720	35.4	28
YuNet faces, shrunk to 640 wide	9.9	101
Haar cascade faces, full 1280 x 720	218.6	5
SFace embedding (per face)	19.2	52

Four optimizations had the largest effect. First, the detector input size is selectable: 320 pixels cut detection time from 46 to 13 milliseconds at the accuracy cost shown in Table II, which is acceptable when buses and people are close to the camera. Second, YuNet cost grows with the number of pixels, so frames are shrunk to 640 pixels on the longest side before face detection, which reduced its time from 35 to 10 milliseconds; boxes and landmarks are scaled back so that alignment still uses the full-resolution face. Third, the object branch can run on every Nth frame while faces are updated on every frame, because objects in view change more slowly than a moving face. With a 320-pixel input and N equal to 2, the estimated frame time falls from 75 to 36 milliseconds for one face. Fourth, the refiner runs only on bus and truck boxes, so its 6 millisecond cost is paid only when a large vehicle is present. YOLOv10-S was not adopted because it gained 6.6 mAP points but was 2.2 times slower. On the accuracy side, the change from a pairwise to a best-match impostor distribution was the most important optimization, and adding truck boxes to the refiner recovered the missed school bus.

VII. Limitations and Ethical Considerations

The school bus and cell phone tests used only ten positive photographs each, so their percentages are coarse, and ImageNet school buses are mostly North American yellow buses. The face results come from LFW, which contains mostly frontal photographs of public figures; Gururaj et al. note that pose, illumination, and aging remain difficult [7], and the robustness test confirmed that both detectors weaken beyond a 30 degree tilt. Accuracy may also differ across demographic groups, which this evaluation did not measure. Timings come from a virtual machine and should be repeated on the demonstration laptop with the included benchmark script.

Face recognition also raises privacy and security concerns. Laishram et al. warn against identifying people without consent [9], so the application recognizes only people who enroll themselves and stores embeddings rather than photographs. The system has no presentation attack detection, so a printed photograph or a screen could be accepted, which is one of the attack vectors reviewed by Leyva et al. [10]. It should therefore not be used to unlock anything or to make decisions about people.

VIII. Conclusion

The application recognizes 80 kinds of objects, distinguishes school buses from other buses, detects faces under rotation and occlusion, and names enrolled people on an ordinary processor with three Python packages. The evaluation showed that lightweight models are accurate enough for this task, that input size and frame skipping are effective ways to trade accuracy for speed, and that three-sigma control limits work well as a face threshold only when they are computed on the same best-match score that the application uses. Future work will add presentation attack detection, test the system on more diverse faces, and replace the fixed class list with an open-vocabulary detector such as YOLO-World [3].

References

[1] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680-1716, Nov. 2023, doi: 10.3390/make5040083.

[2] A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” in *Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 107984-108011, doi: 10.52202/079017-3429.

[3] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-time open-vocabulary object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2024, pp. 16901-16911.

[4] Y. Feng, S. Yu, H. Peng, Y.-R. Li, and J. Zhang, “Detect faces efficiently: A survey and evaluations,” *IEEE Trans. Biom. Behav. Identity Sci.*, vol. 4, no. 1, pp. 1-18, Jan. 2022, doi: 10.1109/TBIOM.2021.3120412.

[5] W. Wu, H. Peng, and S. Yu, “YuNet: A tiny millisecond-level face detector,” *Mach. Intell. Res.*, vol. 20, no. 5, pp. 656-665, Oct. 2023, doi: 10.1007/s11633-023-1423-y.

[6] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, and D. Wen, “SFace: Sigmoid-constrained hypersphere loss for robust face recognition,” *IEEE Trans. Image Process.*, vol. 30, pp. 2587-2598, 2021, doi: 10.1109/TIP.2020.3048632.

[7] H. L. Gururaj, B. C. Soundarya, S. Priya, J. Shreyas, and F. Flammini, “A comprehensive review of face recognition techniques, trends, and challenges,” *IEEE Access*, vol. 12, pp. 107903-107926, 2024, doi: 10.1109/ACCESS.2024.3424933.

[8] M. R. Dileep and A. Danti, “Human age and gender prediction based on neural networks and three sigma control limits,” *Appl. Artif. Intell.*,

vol. 32, no. 3, pp. 281-292, 2018, doi: 10.1080/08839514.2018.1451217.

[9] L. Laishram, M. Shaheryar, J. T. Lee, and S. K. Jung, “Toward a privacy-preserving face recognition system: A survey of leakages and solutions,” *ACM Comput. Surv.*, vol. 57, no. 6, Art. no. 147, pp. 1-38, Feb. 2025, doi: 10.1145/3673224.

[10] R. Leyva, G. Epiphaniou, and C. Maple, “Attack vectors for face recognition systems: A comprehensive review,” *ACM Comput. Surv.*, vol. 58, no. 1, Art. no. 19, pp. 1-37, 2025, doi: 10.1145/3736753.

[11] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 1314-1324, doi: 10.1109/ICCV.2019.00140.

[12] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?” in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, Long Beach, CA, USA, 2019, pp. 5389-5400.